Project 2:

Due Date: 05/01

Points: TBD

Rubric: TBD (Please start working on the project. Once I have set the final exam points, I will assign the total points and rubric for Project 2. It should be around 300 points, but this may change a bit. **The points mentioned in Moodle may change in the future as stated here**.)

Objective

The goal of this project is to refactor your document RAG from Project 1. You will evolve your functional prototype into a robust, maintainable, and testable software application by applying core software design principles (SOLID), documenting your architecture with UML, implementing a test suite, and documenting your work professionally.

Background

In Project 1, you successfully built document RAG, a tool that helps get LLM based answers about your local documents. In the future (not for this course) you may use a local LLM like LLaMA3.1 so that you do not have to send the document to a generative AI company due to various reasons like privacy or trade secrets, etc. This "proof of concept" focused on functionality.

Now, we shift our focus to *software quality*. In the real world, code is read far more often than it is written. It must be flexible to changes (e.g., swapping models, adding new documents, changing the vector DB, etc.) and verifiable (i.e., testable). This project will have you "pay down the technical debt" of Project 1 by refactoring your code to meet these professional standards.

Core Requirements

You must modify your existing Project 1 codebase to meet the following specifications.

1. Implement at least TWO SOLID Principles

You must refactor your code to clearly and correctly implement at least two of the five SOLID principles.

- **(S) Single Responsibility Principle:** A class should have only one reason to change.
- **(O) Open/Closed Principle:** Software entities should be open for extension but closed for modification.

- **(L) Liskov Substitution Principle:** Subtypes must be substitutable for their base types.
- **(I) Interface Segregation Principle:** Clients should not be forced to depend on interfaces they do not use.
- **(D) Dependency Inversion Principle:** High-level modules should not depend on low-level modules. Both should depend on abstractions.

2. Create a UML Class Diagram

You must create a UML Class Diagram that accurately reflects the new, refactored architecture of your application. This diagram should clearly show:

- Your classes
- Their key attributes and methods
- The relationships between them (e.g., association, inheritance, dependency).
- It should be created with some software. (suggestion can use draw.io)
- NO HAND DRAWN IMAGE WILL BE ACCEPTED

3. Create a UML Sequence Diagram

You must create a UML Sequence Diagram to model the dynamic behavior in your application. Your diagram should show the sequence of method calls between your objects, starting from the initial user request to the final returned message. It should be created with some software. (suggestion can use draw.io, Mermaid online editor). NO HAND DRAWN IMAGE WILL BE ACCEPTED.

4. Write Tests

You must write a suite of tests for your application.

- This is now possible because of your SOLID refactoring.
- Use "mock" external dependencies when needed. (e.g. database, API call, etc)
- You **must** also demonstrate how to test a class that *uses* an external service (like Gen AI) by mocking the interface (e.g., creating a MockService that returns fake data without making an API call). Same as above.

5. Professional Documentation

You must provide two forms of documentation:

- **A README.md file:** This file is for a *new developer* joining your project. It must clearly explain:
    - What the project does.
    - How to set up the environment and install dependencies (e.g., pip install -r requirements.txt).
    - How to run the application.
    - How to run your test suite.
- **A REFACTORING.md file:** This is a brief report for me/TA. It must explicitly state:
    - **Which two (or more) SOLID principles** you chose.
    - **Why** you chose them (i.e., what problem in your original code they solved).

- **How** you implemented them. A "before and after" code snippet is highly effective here to demonstrate your changes.

6 Dockerize Your Application

Containerize your application using Docker.

1. Write a Dockerfile that packages your application, its dependencies, and any necessary configuration.
2. Should be able to run the application from a container.
3. Your README.md file should include the commands needed to build the Docker image and run the container.

6 CI pipeline for testing: using Github Actions

1. Set up GitHub Actions to run test cases whenever a developer pushes a new code.

Deliverables

You will submit a link to your private (in future it will be public) Git repository. The repository's main branch must contain:

1. All your refactored source code.
2. Your UML Class Diagram (e.g., as a .png or .svg file).
3. Your UML Sequence Diagram (e.g., as a .png or .svg file).
4. Your test files (e.g., in a /tests directory).
5. Your README.md file.
6. Your REFACTORING.md file.
7. A requirements.txt file.
8. Your Dockerfile.
9. If i missed something that file too!!

RUBRIC: The breakdown of points will be posted soon. BUT it will follow the 6 core requirements given above. The most important point is as follows:

There will be an online one-on-one interview from 05/02 to 05/08 (exact date and time will be conveyed soon). It will be a multiple-day evaluation, and you'll have to pull from dockerhub to run the code. Please do not update GitHub after the deadline, otherwise, I/TA will deduct points. If you miss the interview, no points will be given, even if the software is working. I/TA will ask questions related to the project. You have to upload the project to GitHub and make it private. On the day of the interview, you MUST make it public.

If you have any doubt please ask me during the class time, it may help other students too.